

MVC, Model binding, TagHelpers, HTML Helper classes, View Categorization based on Model Validations Overview , Validations with Data Annotation , Server Side and Client Side Validation , Custom Server side validation , Model level validation using IValidatableObject , Custom unobstrive Client side Validation, Remote Validation, Url Routing Overview, Custom Routes, Attribute Routing , Routing Constraints

What are different types of Web templates available in [ASP.NET](#) Core ?

Which one is best suited for Enterprise application development and Why ?(5-6 reasons)

What is the difference between one way and two data binding ?

What are the differences between get and post methods of HTTP ?

“HTTP is a stateless protocol” Explain Why !!

We have to take care of State management with HTTP

- Client side state management
- Servier Side state management

Why validation is important for a web application WRT to Validation types in an MVC application ?

<https://www.youtube.com/watch?v=x9XWxD6cJuY>

1.Why MVC ? Why is Layered architecture so important in enterprise Application development ?

1. Separation of concerns into :
 - a. Model (Data)
 - b. View (UI)
 - c. Controller (Logic)
2. Full-stack framework (UI + Business Logic + Data)

Use Case: Enterprise applications, large scalable systems

2.Why Razor pages, Where can we use them as a web solution for a client project ?

1. Page-based development model
2. Combines UI + backend logic in one place
3. **Use Case:** Small to medium applications, rapid development (RAD model in SDLC)

3. Why is Web API an ideal solution for complex applications?

1. Backend-only template
2. Returns data (JSON/XML), no UI
3. **Use Case:** Microservices, mobile apps, frontend frameworks like React and Angular

4. Blazor App (WebAssembly/server)

1. C# based frontend (no JavaScript required)
2. **Use Case:** Modern interactive UI apps

5. Minimal APIs

- Lightweight API development
- Minimal configuration
- **Use Case:** Microservices, quick APIs

2. Which Template is Best for Enterprise Applications and Why?

Recommended: MVC + Web API Hybrid Architecture

1. **Separation of Concerns**
 - a. Clear division between UI, business logic, and data
 2. **Scalability**
 - a. Easy to scale modules independently
 3. **Maintainability**
 - a. Code is structured and easier to manage
 4. **Testability**
 - a. Supports unit testing and TDD
 5. **Flexibility with Frontend**
 - a. Can integrate with Angular/React
 6. **Security**
 - a. Built-in authentication and authorization support

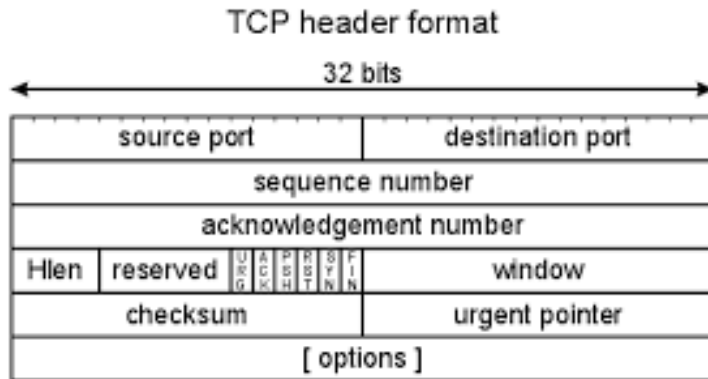
One Way Data binding

Two Way data binding

Data flows in one direction (Model → View)	Data flows both ways (UI ↔ Model)
Displaying data of a student in a view	Form input
@Model.Name	<input asp-for="Name" />

Feature	GET	POST
Purpose	Retrieve data	Send data
Data Location	URL	Request Body
Security	Less secure	More secure
Idempotent(Self join)	Yes	No
Caching	Supported	Not supported
Length Limit	Limited	No limit

Version	IHL	Type of Service	Total Length	
Identification			Flags	Fragment Offset
Time to Live	Protocol		Header Checksum	
Source Address				
Destination Address				
Options				Padding



HTTP does not retain information about previous requests.

Why Stateless?

- Each request is independent
- Server does not remember previous interactions
- Improves scalability and simplicity

Example

- Login request → Server authenticates
- Next request → Server does not remember login

State Management in Web Applications

Client Side Statement management	Server Side state management
1. Cookies 2. Local Storage 3. Session Storage 4. Hidden Fields	1. Session State 2. Application State 3. TempData / ViewData / ViewBag (MVC specific)
• Reduces server load • Faster access	• More secure • Centralized control

1. Less secure
2. Limited storage on client machines(browser)

1. Increases server load
2. Requires memory management at memory

So the ideal way of using Statement management in [ASP.NET](#) Core is a combination of both client side and server side statement management. which is considered a more balanced way of statement management.

Why is Validation imp in a web Application ?

Validation ensures that user input is correct, safe, and usable.

Note: SQL Injection attack that happens in a web application if data is not properly validated for input. `1 OR 1' = 1` that returns true hence mimicking a Query Output

V.Imp for VIVA

Why Validation is Important

1. Prevents invalid data entry
2. Protects against security attacks (e.g., SQL Injection)
3. Improves user experience
4. Ensures data consistency
5. Reduces application errors

Types of Validation in MVC Applications

Client-Side Validation • Happens in browser • Uses JavaScript	Server-Side Validation • Happens on server after submission • More secure	Data Annotation Validation (User Model Class) ex <pre>public class UserModel { [Required] public string Name { get; set; } [Range(18, 60)] public int Age { get , set} }</pre>
4. Custom Validation • Based on business rules		

Scenario Overview

A college wants to build a simple student registration portal using ASP.NET Core where students can enter their name and receive a confirmation message instantly.(so we have to implement here 2- way data binding)

The system should:

- Accept student input (Name)
- Process the request

- Display a personalized response along with the details

Business Requirement(BR Document)

The college admin requires:

- A lightweight web page
- Minimal backend complexity
- Automatic data handling without manual parsing

Use Razor Pages with Two-Way Data Binding

Acceptance Criteria

- User should be able to enter name
- Data should be automatically captured in backend
- System should display: “Hello <Name>”
- No manual data extraction from request

We will use:

- Razor Pages
- `[BindProperty]` for two-way binding
- `OnPost()` handler

Step 1: Create a [ASP.NET](#) Web APP razor page application with Page model as below (backEnd Logic)([.cshtml.cs](#) file)

```
[BindProperty]
public string StudentName { get; set; }

public string Message { get; set; }
public void OnPost()
{
    Message = "Welcome " + StudentName + " to the portal!";
}
```

```
}
```

Step 2: Draft UI page with below logic(.cshtml file)

```
@page
@model StudentPortal.Pages.IndexModel

<h2>Student Registration</h2>

<form method="post">
<label>Enter Name:</label>
<input asp-for="StudentName" /> ( for implementing 2 way data binding)
<button type="submit">Register</button>
</form>

@if (Model.Message != null)
{
<p>@Model.Message</p>
}
```

General flow of action items in above case study

1. Student enters name in textbox
2. Clicks Register
3. Razor Pages automatically binds input → **StudentName**
4. **OnPost()** executes
5. Message generated
6. UI displays response

Scenario Overview

A company wants to build a **simple employee feedback portal** using ASP.NET Core.

The system should:

- Display employee details (read-only)
- Allow employees to submit feedback
- Show confirmation after submission

One-Way Binding (Display data)

Two-Way Binding (Form input)

Tag Helpers (asp-for, asp-validation-for)

HR team wants:

- Employee name displayed automatically
- Feedback form submission
- Clean UI with minimal backend code

Acceptance Criteria

- Employee name should be displayed (read-only)
- Feedback should be entered via form
- System should display confirmation message
- Validation should prevent empty submission

Step 1: Implement page Model(backEnd Code)

Step 2: Implement UI (Index.cshtml)

Flow Explanation

One-Way Binding Flow

- Backend → UI
- `EmployeeName` displayed using `@Model.EmployeeName`

Two-Way Binding Flow

- UI → Backend → UI

1. User enters feedback
2. `asp-for` binds input to `Feedback` property
3. `[BindProperty]` captures value
4. `OnPost()` processes
5. Message displayed

Tag Helper	Purpose
<code>asp-for</code>	Binds input to model property
<code>asp-validation-for</code>	Displays validation errors

Case Study: Online Course Registration Portal

Scenario:

A training institute is building a portal using ASP.NET Core where users can:

- Register for courses
- Validate inputs securely
- Navigate using clean URLs

Step 1: Create a new project with MVC template and name it as Course registration Portal.

Step 2: Create [Student.cs](#) inside the Model folder with all required properties.

Step 3: Create Student Controller inside Controller folder

Step 4: Create a view with UI + tag helpers

Step 5: Enable validation in `_layout.cshtml`

Flow Summary

1. User opens form → Routing directs request
2. User enters data → Tag Helpers bind data
3. Client-side validation triggers
4. Data sent to server
5. Server-side validation (`ModelState`) checks
6. Response returned

IMP take away from the demo

- Data Annotation → Validation rules
- Tag Helpers → UI binding
- ModelState → Server validation
- Routing → URL mapping